CSE110: Programming Language I (Lab)
Spring 2026
Lab Exam - 02
Duration: 25 Minutes

A

Name:	ID:	Section:

Question 1 [CO1] [10 Points]

Write a Java program to analyze food sales during the celebration of Pahela Baishakh at BRAC University. The program should ask the user to enter the number of student stalls in the festival. For each student stall, the number of items sold is equal to the stall number. For example, Stall 1 sells 1 item, Stall 2 sells 2 items, Stall 3 sells 3 items, and so on. For each item, the user must input its price.

For every student stall, the program should display all items that cost more than 200 taka (premium items) and calculate the total sales of that stall. If a stall has 2 or more premium items, the program should print "**Popular Stall!**". After processing all stalls, the program should display the overall total sales of the festival.

Input	Output
Enter number of stalls: 2 Stall 1: Enter price of item 1: 220 Stall 2: Enter price of item 1: 150 Enter price of item 2: 250	Stall 1: Premium items: 220 Total sales: 220 Stall 2: Premium items: 250 Total sales: 400 Overall Festival Sales: 620
Enter number of stalls: 3 Stall 1: Enter price of item 1: 150 Stall 2: Enter price of item 1: 250 Enter price of item 2: 300 Stall 3: Enter price of item 1: 100 Enter price of item 2: 220 Enter price of item 3: 50	Stall 1: Premium items: Total sales: 150 Stall 2: Premium items: 250 300 Total sales: 550 **Popular Stall!** Stall 3: Premium items: 220 Total sales: 370 Overall Festival Sales: 1070

CSE110: Programming Language I (Lab)
Spring 2026
Lab Exam - 02
Duration: 25 Minutes

B

Name:	ID:	Section:

Question 1 [CO1] [10 Points]

Write a Java program to analyze cultural sessions during the celebration of Pahela Baishakh at BRAC University. The program should ask the user to enter the number of sessions. For each session, the program should first take input of the number of performances. Then, for each performance, the user must input its duration (in minutes).

For every session, the program should count how many performances have a duration greater than 30 minutes and calculate the total duration of that session. If more than half of the performances in a session have a duration greater than 30 minutes, the program should print "**Long Session!**". After processing all sessions, the program should display the total duration of all sessions combined.

Input	Output
Enter number of sessions: 1 Session 1: Enter number of performances: 4 Enter duration of performance 1: 10 Enter duration of performance 2: 35 Enter duration of performance 3: 45 Enter duration of performance 4: 20	Session 1: Long performances count: 2 Total duration: 110 Overall Total Duration: 110
Enter number of sessions: 2 Session 1: Enter number of performances: 3 Enter duration of performance 1: 20 Enter duration of performance 2: 40 Enter duration of performance 3: 50 Session 2: Enter number of performances: 2 Enter duration of performance 1: 25 Enter duration of performance 2: 30	Session 1: Long performances count: 2 Total duration: 110 **Long Session!** Session 2: Long performances count: 0 Total duration: 55 Overall Total Duration: 165

CSE110: Programming Language I (Lab)
Spring 2025
Lab Exam - 03
Duration: 25 Minutes

A

Name:	ID:	Section:

Question 1 [CO2] [10 Points]

Write a Java program that reads a string from the user and produces its run-length encoded form. In run-length encoding, each group of consecutive identical characters is replaced by the character followed by its count. If a character appears only once in a row, write the character without a number. After producing the encoded string, compare its length with the original. If the encoded version is shorter, print the encoded string. Otherwise, print the original string unchanged.

Sample Input	Sample Output
aabbbccddddee	a2b3c2d4e2
abcd	abcd (encoded "a1b1c1d1" is longer, so print original)
aaabbaaa	a3b2a3

CSE110: Programming Language I (Lab)
Spring 2025
Lab Exam - 03
Duration: 25 Minutes

B

Name:	ID:	Section:

Question 1 [CO2] [10 Points]

Write a Java program that reads a main string and a pattern string, then prints how many times the pattern appears in the main string (non-overlapping occurrences only). You must implement the search manually, no built-in method may be used to find or detect the substring.

Sample Input	Sample Output
main: ababab pattern: ab	Count: 3
main: aaaaaa pattern: aa	Count: 3
main: hello world pattern: xyz	Count: 0

CSE110: Programming Language I (Lab)
Spring 2026
Lab Exam - 04
Duration: 25 Minutes

A

Name:	ID:	Section:

Question 1 [CO3] [10 Points]

A weekly attendance system stores 7 digits in an array, where **each index** represents a day of the week in order from Saturday to Friday. In this system, classes on **Saturday** (index 0) and **Tuesday** (index 3) are conducted **online**, while all other days are conducted **in-person**. A day is considered balanced if the number of elements before it (to the left) with lower values is equal to the number of elements after it (to the right) with higher values.

Write a Java program to determine and print **all** balanced days **only among the in-person days**, and also print the **total** number of such balanced in-person days.

Given Array	**Output**
`int[] arr = {40, 10, 20, 100, 15, 30, 5};`	`Balanced in-person days are: 2 4 6` `Total balanced in-person days: 3`

Explanation:

Monday (index **2**, value 20): left lower count = 1 (10), right higher count = 1 (30), ignoring indices 0 and 3 (online). Since both are equal, the day is balanced.

Wednesday (index **4**, value 15): left lower count = 1 (10), right higher count = 1 (30), ignoring indices 0 and 3 (online). Since both are equal, the day is balanced.

Friday (index **6**, value 5): left lower count = 0, right higher count = 0, ignoring indices 0 and 3 (online). Since both are equal, the day is balanced.

CSE110: Programming Language I (Lab)
Spring 2026
Lab Exam - 04
Duration: 25 Minutes

B

Name:	ID:	Section:

Question 1 [CO3] [10 Points]

A FIFA World Cup performance tracker stores the number of goal chances created for 7 matches in an array, where **each index** represents a match in sequence from Match 1 to Match 7. In this system, matches played on **Monday** (index 1) and **Thursday** (index 4) are against **rival teams**, while all other matches are **non-rival**. A match is considered Growth-Stable if the number of non-rival matches before it (to the left) with higher values is equal to the number of non-rival matches after it (to the right) with lower values.

Write a Java program to determine and print **all** Growth-Stable matches **only among the non-rival matches**, and also print the **total** number of such Growth-Stable matches.

Given Array	**Output**
`int[] scores = {40, 10, 20, 25, 15, 30, 5};`	Growth-Stable matches are: 2 3 5 Total Growth-Stable matches: 3

Explanation:

Match 3 (index **2**, value 20): left higher count = 1 (40), right lower count = 1 (5), ignoring indices 1 and 4. Since both are equal, the match is Growth-Stable.

Match 4 (index **3**, value 25): left higher count = 1 (40), right lower count = 1 (5), ignoring indices 1 and 4. Since both are equal, the match is Growth-Stable.

Match 6 (index **5**, value 30): left higher count = 1 (40), right lower count = 1 (5), ignoring indices 1 and 4. Since both are equal, the match is Growth-Stable.